

MODULE 1

OOA (Object Oriented Analysis)

Definition:

Process of identifying objects, classes, attributes and relationships from a problem before coding.

OOD (Object Oriented Design)

Definition:

Process of designing classes, methods and relationships after analysis.

OOP (Object Oriented Programming)

Definition:

Programming paradigm that organizes software around objects rather than functions.

Class

Definition:

A class is a blueprint for creating objects.

Example:

```
class Student
{
}
```

Object

Definition:

An object is an instance of a class.

Example:

```
Student s=new Student();
```

Four Pillars of OOP

Encapsulation

Definition:

Wrapping data and methods together into a single unit.

Advantage:

Data Security

Abstraction

Definition:

Showing important details and hiding implementation details.

Example:

Car driving

Inheritance

Definition:

Process of acquiring properties and methods of another class.

Advantage:

Code Reusability

Polymorphism

Definition:

One interface, many forms.

Example:

Method Overloading

Relationships among Objects and Classes

Association

Definition:

Relationship between two independent classes.

Example:

Teacher ---- Student

Aggregation

Definition:

Weak HAS-A relationship.

Example:

Department ☐---- Professor

Professor can exist independently.

Composition

Definition:

Strong HAS-A relationship.

Example:

House ☐---- Room

Room cannot exist independently.

Generalization

Definition:

Combining common properties into a parent class.

Dog

Cat

↓

Animal

Specific → General

Specialization

Definition:

Creating specialized classes from a parent class.

Animal

↓

Dog

Cat

General → Specific

Link

Definition:

An instance of association.

Example:

Student ---- Teacher

Actual connection:

Rahul ---- Sharma Sir

Meta Class

Definition:

A class whose objects are themselves classes.

Java Features

Simple

Easy syntax.

Object Oriented

Based on classes and objects.

Platform Independent

Write Once Run Anywhere.

Secure

No pointers.

Robust

Strong exception handling.

Portable

Runs on different platforms.

Multithreaded

Supports multiple threads.

Distributed

Supports network applications.

High Performance

Uses JIT compiler.

Dynamic

Classes loaded at runtime.

Java vs C++

Java

- Platform Independent
- JVM
- No Pointers
- Garbage Collection

C++

- Platform Dependent
 - No JVM
 - Supports Pointers
 - Manual Memory Management
-

MODULE 2

Java Fundamentals

JVM

Definition:

Java Virtual Machine executes Java bytecode.

Bytecode

Definition:

Intermediate code generated by Java compiler.

Extension:

.class

Java Execution Process

.java



Compiler



.class



JVM



Output

Data Types

Primitive Data Types

byte

short

int

long

float

double

char

boolean

Non Primitive Data Types

String

Array

Class

Object

Interface

Variable

Definition:

A named memory location used to store data.

Local Variable

Declared inside method.

Instance Variable

Declared inside class but outside methods.

Static Variable

Shared by all objects.

Operator

Definition:

Symbol used to perform operations.

Arithmetic Operator

+

-

*

/

%

Relational Operator

==

!=

>

<

>=

<=

Returns:

true
false

Logical Operator

&&
||
!

Assignment Operator

=
+=
-=
*=
/=

Increment Operator

++

Decrement Operator

--

Control Statements

if

Executes block if condition is true.

if-else

Selects one of two blocks.

switch

Selects one option from multiple choices.

Loops

for Loop

Used when iterations are known.

while Loop

Executes while condition is true.

do-while Loop

Executes at least once.

Array**Definition:**

Collection of similar type elements stored in contiguous memory locations.

OOP in Java

Method**Definition:**

A block of code that performs a specific task.

Constructor

Definition:

Special method used to initialize objects.

Characteristics:

- Same name as class
 - No return type
 - Called automatically
-

Default Constructor

Constructor without parameters.

Parameterized Constructor

Constructor with parameters.

Constructor Overloading**Definition:**

Multiple constructors with different parameter lists.

Method Overloading**Definition:**

Multiple methods with same name but different parameters.

this Keyword**Definition:**

Refers to current object.

Uses:

- Current variable
- Current method

- Current constructor

Garbage Collection

Garbage

Unused object.

Garbage Collection

Definition:

Process of removing unused objects from memory.

Garbage Collector

JVM component that removes garbage.

finalize()

Definition:

Method executed before object destruction.

Parameter Passing

Call By Value

Copy of variable is passed.

Call By Reference

Reference/address is passed.

Java Reality

Java supports only:

Call By Value

Static

static Keyword

Creates class-level members.

Static Variable

One copy shared by all objects.

Static Method

Method that can be called without object.

Example:

```
Test.show();
```

Nested Class

Definition:

Class declared inside another class.

Inner Class

Definition:

Non-static class declared inside another class.

MOST IMPORTANT DIFFERENCES

Aggregation vs Composition

Aggregation Composition

Weak HAS-A Strong HAS-A

Independent Dependent

Generalization vs Specialization

Generalization Specialization

Specific→General General→Specific

Constructor vs Method

Constructor Method

Initializes Object Performs Task

Same Name as Class Any Name

Method Overloading vs Constructor Overloading

Method Constructor

Multiple Methods Multiple Constructors

JVM vs Bytecode

JVM Bytecode

Executes Code Generated Code

Call By Value vs Call By Reference

Value	Reference
-------	-----------

Copy Passed	Address Passed
-------------	----------------

PART 1 COMPLETE

MEGA REVISION PART 2

MODULE 3 + MODULE 4

MODULE 3

String

String

Definition:

A string is a sequence of characters enclosed within double quotes.

Example:

```
String s="Java";
```

Mutable and Immutable String

Immutable String

Definition:

A string whose value cannot be changed after creation.

Example:

```
String s="Java";
```

String class is immutable.

Mutable String

Definition:

A string whose value can be changed after creation.

Example:

```
StringBuffer sb=  
new StringBuffer("Java");
```

String Class Methods

charAt()

Returns character at specified index.

compareTo()

Compares two strings lexicographically.

Returns:

0

Positive

Negative

equals()

Compares contents of two strings.

Returns:

true

false

equalsIgnoreCase()

Compares strings ignoring case.

Example:

JAVA

java

Result:

true

indexOf()

Returns position of first occurrence.

length()

Returns length of string.

substring()

Returns part of string.

toCharArray()

Converts string into character array.

toLowerCase()

Converts string into lowercase.

toUpperCase()

Converts string into uppercase.

trim()

Removes leading and trailing spaces.

toString()

Returns string representation.

valueOf()

Converts data into String.

Example:

```
String.valueOf(100);
```

Result:

"100"

StringBuffer

StringBuffer

Definition:

A mutable string class.

StringBuffer Methods

append()

Adds data at end.

insert()

Inserts data at specified position.

delete()

Deletes characters.

deleteCharAt()

Deletes character at specified position.

reverse()

Reverses string.

replace()

Replaces characters.

capacity()

Returns current capacity.

charAt()

Returns character at index.

ensureCapacity()

Increases capacity.

getChars()

Copies characters into array.

indexOf()

Returns first occurrence position.

length()

Returns number of characters.

setCharAt()

Changes character at specified position.

setLength()

Changes length of StringBuffer.

substring()

Returns part of StringBuffer.

toString()

Converts StringBuffer into String.

Difference: String vs StringBuffer

String	StringBuffer
Immutable	Mutable
New Object Created	Same Object Modified
Slower	Faster

Command Line Argument

Definition:

Values passed during program execution.

Example:

java Test Rahul

Rahul = command line argument

BufferedReader

Definition:

Class used for efficient character input.

Package:

java.io

Method:

readLine()

Scanner

Definition:

Class used to take user input.

Package:

java.util

Methods:

nextInt()

nextFloat()

nextDouble()

next()

nextLine()

Difference: BufferedReader vs Scanner

BufferedReader Scanner

Faster

Slower

Complex

Easy

readLine()

nextInt() etc

MODULE 4

Inheritance

Definition:

Process of acquiring properties and methods of another class.

Keyword:

extends

Parent Class

Class whose properties are inherited.

Child Class

Class that inherits properties.

Types of Inheritance

Single Inheritance

A

↓

B

Multilevel Inheritance

A

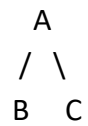
↓

B

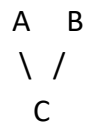
↓

C

Hierarchical Inheritance



Multiple Inheritance



Not supported by classes.

Supported by interfaces.

Hybrid Inheritance

Combination of inheritance types.

Supported through interfaces.

Advantages of Inheritance

- Code Reusability
- Easy Maintenance
- Less Duplication
- Hierarchical Organization

super Keyword

Definition:

Refers to immediate parent class object.

Uses:

- Parent Variable
 - Parent Method
-

super()

Definition:

Invokes parent class constructor.

final Keyword

Definition:

Restricts modification.

final Variable

Value cannot be changed.

final Method

Cannot be overridden.

final Class

Cannot be inherited.

Method Overriding

Definition:

Redefining parent class method in child class using same signature.

Requirements:

- Inheritance
- Same Method Name
- Same Parameters

Dynamic Method Dispatch

Definition:

Calling overridden method at runtime through parent class reference.

Example:

```
A obj = new B();
```

Runtime decides method.

Abstract Class

Definition:

Class whose objects cannot be created.

Keyword:

abstract

Abstract Method

Definition:

Method without body.

Example:

```
abstract void show();
```

Important Rule

If a class contains abstract method,
then class must be abstract.

Interface

Definition:

Collection of abstract methods.

Keyword:

interface

Implementing Interface

Keyword:

implements

Uses of Interface

- Abstraction
 - Multiple Inheritance
 - Standard Rules
-

Multiple Inheritance Using Interface

Example:

```
class C implements A,B
{
}
```

Difference: Abstract Class vs Interface

Abstract Class	Interface
abstract keyword	interface keyword
Can have normal methods	Abstract methods
extends	implements
Partial abstraction	Full abstraction

Package

Definition:

Collection of related classes and interfaces.

Advantages of Package

- Organization
 - Reusability
 - Security
 - Avoids Name Conflict
-

Creating Package

Keyword:

```
package mypack;
```

Importing Package

Example:

```
import java.util.Scanner;
```

Import Entire Package

```
import java.util.*;
```

Access Modifiers**Definition:**

Keywords used to control accessibility of members.

public

Accessible everywhere.

private

Accessible only within class.

protected

Accessible within package and inheritance.

default

Accessible only within package.

Access Modifier Table ☆☆☆

Modifier	Same Class	Same Package	Subclass	Other Package
----------	------------	--------------	----------	---------------

public	✓	✓	✓	✓
--------	---	---	---	---

protected	✓	✓	✓	X
-----------	---	---	---	---

default	✓	✓	X	X
---------	---	---	---	---

private	✓	X	X	X
---------	---	---	---	---

Memorize this.

Member Access for Packages

Definition:

Accessibility of package members depends on access modifiers.

MOST IMPORTANT DIFFERENCES

String vs StringBuffer

String StringBuffer

Immutable Mutable

BufferedReader vs Scanner

BufferedReader Scanner

Faster Easier

this vs super

this super

Current Object Parent Object

Overloading vs Overriding

Overloading Overriding

Compile Time Runtime

Same Class Parent + Child

Abstract Class vs Interface

Abstract Class Interface

Partial Abstraction Full Abstraction

final Method vs Method Overriding

final Method Overriding

Cannot Override Redefines Method

PART 2 COMPLETE

You have now revised:

- ✓ Module 1
- ✓ Module 2
- ✓ Module 3
- ✓ Module 4

okay go ahead

MEGA REVISION PART 3

MODULE 5 (Exception Handling + Multithreading + Applet)

This module is extremely important because many direct theory questions come from it.

EXCEPTION HANDLING

Exception

Definition:

An unwanted event that occurs during program execution and interrupts the normal flow of the program.

Example:

10/0

Produces:

ArithmeticException

Exception Handling

Definition:

The process of handling runtime errors so that the program continues execution.

Why Exception Handling?

- Prevents program termination
- Improves reliability
- Handles errors gracefully

Types of Exceptions

Checked Exception

Definition:

Exception checked by compiler at compile time.

Examples:

IOException

SQLException

ClassNotFoundException

Unchecked Exception

Definition:

Exception checked during runtime.

Examples:

ArithmeticException

NullPointerException

ArrayIndexOutOfBoundsException

Difference: Checked vs Unchecked

Checked	Unchecked
---------	-----------

Compile Time	Runtime
--------------	---------

Must Handle	Not Mandatory
-------------	---------------

Exception Hierarchy ☆

Object



Throwable

└─ Error

└─ Exception

 └─ IOException

 └─ SQLException

 └─ RuntimeException

 └─ ArithmeticException

 └─ NullPointerException

 └─ ArrayIndexOutOfBoundsException

try

Definition:

Contains code that may generate exception.

catch

Definition:

Handles exceptions generated in try block.

finally

Definition:

Block that executes whether exception occurs or not.

throw

Definition:

Used to explicitly generate an exception.

Example:

```
throw new ArithmeticException();
```

throws

Definition:

Used to declare exceptions in method declaration.

Example:

```
void show() throws Exception
{
}
```

Difference: throw vs throws ☆☆☆

throw	throws
Inside Method	Method Declaration
Generates Exception	Declares Exception

User Defined Exception

Definition:

Custom exception created by programmer.

Example:

```
class AgeException
extends Exception
{
}
```

MULTITHREADING

Thread

Definition:

Smallest unit of execution within a process.

Multithreading

Definition:

Concurrent execution of two or more threads.

Advantages

- Better CPU Utilization
 - Faster Execution
 - Better Responsiveness
 - Improved Performance
-

Thread Life Cycle ☆☆☆☆☆

New



Runnable



Running



Waiting/Blocked



Runnable



Terminated

Memorize this.

New State

Thread object created.

Thread t=new Thread();

Runnable State

After:

```
t.start();
```

Waiting for CPU.

Running State

CPU starts execution.

Waiting/Blocked State

Thread waits using:

```
sleep()
```

```
wait()
```

```
join()
```

Terminated State

Execution completed.

Creating Threads

Method 1: Thread Class

Definition:

Thread created by extending Thread class.

Example:

```
class MyThread  
extends Thread  
{  
}
```

Method 2: Runnable Interface

Definition:

Thread created by implementing Runnable interface.

Example:

```
class MyThread
implements Runnable
{
}
```

Difference: Thread vs Runnable

Thread	Runnable
Extend Thread	Implement Runnable
Less Flexible	More Flexible

Thread Priority

Definition:

Priority assigned to thread that influences CPU scheduling.

Priority Range

MIN_PRIORITY = 1

NORM_PRIORITY = 5

MAX_PRIORITY = 10

Important Thread Methods

start()

Starts thread execution.

run()

Contains thread task.

sleep()

Definition:

Temporarily pauses thread.

Example:

```
Thread.sleep(1000);
```

join()

Definition:

Makes one thread wait for another thread.

yield()

Definition:

Allows another thread to execute.

isAlive()

Definition:

Checks whether thread is active.

suspend()

Definition:

Temporarily stops thread.

⚠️ Deprecated.

resume()

Definition:

Restarts suspended thread.

⚠️ Deprecated.

Synchronization

Synchronization

Definition:

Mechanism used to control multiple threads accessing shared resources.

Race Condition

Definition:

Incorrect result caused when multiple threads access shared data simultaneously.

synchronized Method

Example:

```
synchronized void show()  
{  
}  
}
```

Only one thread executes at a time.

Inter Thread Communication

Definition:

Mechanism through which threads communicate and coordinate execution.

wait()

Definition:

Suspends thread and releases lock until notified.

notify()**Definition:**

Wakes one waiting thread.

notifyAll()**Definition:**

Wakes all waiting threads.

Difference: wait() vs sleep() ☆☆☆☆☆

wait()	sleep()
---------------	----------------

Releases Lock	Does Not Release Lock
---------------	-----------------------

Object Class	Thread Class
--------------	--------------

Needs notify()	Auto Resume
----------------	-------------

Very Important.

Difference: notify() vs notifyAll()

notify()	notifyAll()
-----------------	--------------------

One Thread	All Threads
------------	-------------

Deadlock ☆☆☆☆☆**Definition:**

Situation where two or more threads wait indefinitely for each other.

Example

Thread 1
holds A
waits B

Thread 2
holds B
waits A

Result:

Deadlock

Causes of Deadlock

Mutual Exclusion

Only one thread uses resource.

Hold and Wait

Thread holds one resource and waits for another.

Circular Wait

Threads wait in cycle.

Prevention

- Proper Resource Allocation
 - Avoid Circular Wait
 - Resource Ordering
-

APPLET

Applet

Definition:

Small Java program that runs inside a web browser.

Features of Applet

- Browser Based
 - No main()
 - Uses Graphics
 - Event Driven
-

Applet Life Cycle ☆☆☆☆☆

init()

↓

start()

↓

paint()

↓

stop()

↓

destroy()

Memorize exactly.

init()

Definition:

Called once when applet loads.

Purpose:

Initialization.

start()

Definition:

Called after init().

Purpose:

Starts execution.

paint()**Definition:**

Displays output on screen.

stop()**Definition:**

Temporarily stops applet.

destroy()**Definition:**

Releases resources and removes applet.

Application vs Applet ☆☆☆☆**Application****Applet**

Standalone Program Browser Program

Uses main()

No main()

JVM Starts

Browser Starts

Direct Execution

Browser Execution

Parameter Passing in Applet**Definition:**

Parameters are passed using HTML <param> tag.

Example:

```
<param name="msg"  
value="Hello">
```

getParameter()

Used to receive parameter.

```
getParameter("msg");
```

FINAL HIGH-YIELD DEFINITIONS LIST

Exception

Unwanted event during execution.

Checked Exception

Compile-time exception.

Unchecked Exception

Runtime exception.

try

Contains risky code.

catch

Handles exception.

finally

Always executes.

throw

Generates exception.

throws

Declares exception.

User Defined Exception

Programmer-created exception.

Thread

Smallest execution unit.

Multithreading

Executing multiple threads simultaneously.

Thread Priority

CPU preference value.

Synchronization

Controls access to shared resource.

Race Condition

Incorrect result due to simultaneous access.

wait()

Suspends thread and releases lock.

notify()

Wakes one thread.

notifyAll()

Wakes all threads.

Deadlock

Threads waiting forever for each other.

Applet

Browser-based Java program.

init()

Initializes applet.

start()

Starts applet.

paint()

Displays output.

stop()

Stops applet temporarily.

destroy()

Destroys applet.

COMPLETE SYLLABUS STATUS

- ✓ Module 1 Revision Complete
- ✓ Module 2 Revision Complete
- ✓ Module 3 Revision Complete
- ✓ Module 4 Revision Complete
- ✓ Module 5 Revision Complete